AuraChat

Technical Design Document

Architecture, Data Flow, and Implementation Blueprint

Version 1.0 | March 2026 | Author: Patrick

---

1. Document Overview

This Technical Design Document (TDD) defines how AuraChat is built — the architecture decisions, data models, dependency graph, data flow, navigation structure, and key implementation patterns. It is the primary reference for AI-assisted (vibe) coding and should be fed as context when prompting any AI coding agent.

ℹ *Read this document top-to-bottom before writing any code. Every section informs the next.*

---

2. Architecture Overview

2.1 Pattern: Clean MVVM + Repository

AuraChat follows a Clean Architecture approach with three clearly separated layers. No layer can skip another — data always flows through the layer directly above or below it.

Layer	Package	Responsibility	Key Rule
Presentation	presentation/	UI (Compose) + ViewModels	No business logic. No direct DB/API calls.
Domain	domain/	Use Cases + Repository Interfaces	Pure Kotlin. No Android imports.
Data	data/	Room, Firebase AI, Repositories	Implements domain interfaces. No UI imports.

ℹ *The most important rule for vibe coding: if your AI agent puts an API call or database query inside a Composable function, that is wrong. All logic lives in ViewModels and below.*

2.2 Key Architectural Decisions & Trade-offs

Decision 1: Firebase AI Logic over standalone Gemini SDK

Option	Pro	Con	Decision
Firebase AI Logic	Secure — no API key in APK. App Check support. Future-proof.	Requires Firebase project setup, google-services.json.	CHOSEN
Standalone Gemini SDK	Simpler initial setup.	Deprecated Dec 2025. API key hardcoded in APK. Security risk.	Rejected
Google GenAI Java SDK	Non-Firebase alternative.	Server-side design, less Android-optimised. Still needs key in APK.	Rejected

Decision 2: Hilt for Dependency Injection

Option	Pro	Con	Decision
Hilt	Official Google DI. Compile-time safety. ViewModel integration built-in.	Steeper initial setup. AGP 9.0+ required.	CHOSEN
Manual DI	Zero boilerplate to start.	Doesn't scale. Hard to test. Error-prone as app grows.	Rejected
Koin	Simpler syntax. Runtime DI.	Runtime errors instead of compile-time. Less idiomatic for Android.	Rejected

Decision 3: KSP over kapt for annotation processing

Both Hilt and Room use KSP (Kotlin Symbol Processing) instead of the deprecated kapt processor. KSP processes Kotlin directly — roughly 2x faster build times and required for Kotlin 2.x compatibility. No kapt configuration appears anywhere in this project.

Decision 4: Room for local persistence

Chat sessions and messages are stored locally in a Room (SQLite) database. This gives offline access to history, fast queries, and reactive Flow-based updates. No cloud sync is in scope for v1.

Decision 5: StateFlow for UI state

ViewModels expose a single StateFlow<UiState> to Compose. Compose observes this with collectAsStateWithLifecycle(). This prevents multiple recompositions from multiple separate state values and makes state changes predictable and testable.

---

3. Hilt Dependency Graph

3.1 What Hilt Manages

Hilt is responsible for constructing and delivering every major class in AuraChat. The dependency graph below shows what depends on what. Hilt reads this graph at compile time and wires everything automatically.

Class	Scope	Depends On	Provided By
AuraChatDatabase	@Singleton	Application Context	DatabaseModule
ChatSessionDao	@Singleton	AuraChatDatabase	DatabaseModule
ChatMessageDao	@Singleton	AuraChatDatabase	DatabaseModule
FirebaseAIDataSource	@Singleton	Firebase.ai instance	AIModule
ChatRepositoryImpl	@Singleton	ChatSessionDao, ChatMessageDao, FirebaseAIDataSource	RepositoryModule
ChatViewModel	@HiltViewModel	ChatRepository (interface)	@HiltViewModel + @Inject
HomeViewModel	@HiltViewModel	ChatRepository (interface)	@HiltViewModel + @Inject
HistoryViewModel	@HiltViewModel	ChatRepository (interface)	@HiltViewModel + @Inject
SettingsViewModel	@HiltViewModel	SettingsRepository	@HiltViewModel + @Inject

3.2 Hilt Modules

Three Hilt modules handle all dependency provision. Each module is a Kotlin object annotated with @Module and @InstallIn.

DatabaseModule

ℹ *Installed in SingletonComponent — lives for the entire app lifetime.*

- Provides AuraChatDatabase as @Singleton using Room.databaseBuilder()
- Provides ChatSessionDao by calling database.chatSessionDao()
- Provides ChatMessageDao by calling database.chatMessageDao()

AIModule

ℹ *Installed in SingletonComponent.*

- Provides the Firebase GenerativeModel as @Singleton
- Configured with model name 'gemini-2.0-flash' by default
- Backend: GenerativeBackend.googleAI()

- Model name is read from SettingsRepository so user can switch models

RepositoryModule

ℹ *Installed in SingletonComponent. Binds interfaces to implementations.*

- Binds ChatRepository interface → ChatRepositoryImpl
- Binds SettingsRepository interface → SettingsRepositoryImpl
- Use @Binds (not @Provides) since we're binding an interface to its implementation

---

4. Data Layer

4.1 Room Database Schema

Entity: ChatSession

Column	Type	Description
id	String (UUID)	Primary key. Generated on session creation.
title	String	Auto-generated from first user message (truncated to 40 chars).
createdAt	Long (epoch ms)	Timestamp when session was first created.
updatedAt	Long (epoch ms)	Timestamp of most recent message. Used for sort order in drawer.

Entity: ChatMessage

Column	Type	Description
id	String (UUID)	Primary key.
sessionId	String (FK)	Foreign key referencing ChatSession.id.
role	String (enum)	'USER' or 'MODEL'. Determines bubble alignment and colour.
content	String	The text content of the message.
imageUri	String? (nullable)	Local URI string for attached image, null if no image.
timestamp	Long (epoch ms)	When the message was sent/received.
isStreaming	Boolean	True while AI is still streaming. False when complete.

Relationships

- One ChatSession has many ChatMessages (one-to-many)

- ChatMessage.sessionId is a foreign key with CASCADE delete — deleting a session deletes all its messages automatically
- Room query for full session: join ChatSession + ChatMessages ordered by timestamp ASC

4.2 DAO Interfaces

ChatSessionDao

- fun getAllSessions(): Flow<List<ChatSession>> — returns reactive list ordered by updatedAt DESC
- suspend fun insertSession(session: ChatSession)
- suspend fun updateSession(session: ChatSession)
- suspend fun deleteSession(session: ChatSession)
- suspend fun getSessionById(id: String): ChatSession?

ChatMessageDao

- fun getMessagesForSession(sessionId: String): Flow<List<ChatMessage>> — ordered by timestamp ASC
- suspend fun insertMessage(message: ChatMessage)
- suspend fun updateMessage(message: ChatMessage) — used to update streaming message content
- suspend fun deleteMessagesForSession(sessionId: String)

4.3 FirebaseAIDataSource

This class wraps the Firebase AI SDK and exposes a clean Kotlin Flow-based API to the rest of the app. The ViewModel never imports Firebase directly.

- streamResponse(history: List<ChatMessage>, prompt: String, imageBitmap: Bitmap?): Flow<String>
- Internally builds a GenerativeModel chat instance with the history converted to Content objects
- Calls model.generateContentStream(prompt) which returns a Flow<GenerateContentResponse>
- Maps each response chunk to its text content and emits the string tokens downstream
- Image attachment: if imageBitmap is not null, it is included as an InlineDataPart in the request content
- Error handling: catches GoogleGenerativeAIException and emits a descriptive error string rather than crashing

4.4 ChatRepository Interface & Implementation

ChatRepository Interface (domain layer)

- fun getSessions(): Flow<List<ChatSession>>
- fun getMessages(sessionId: String): Flow<List<ChatMessage>>
- suspend fun createSession(): ChatSession

- suspend fun deleteSession(sessionId: String)
- fun streamAIResponse(sessionId: String, history: List<ChatMessage>, prompt: String, image: Bitmap?): Flow<String>
- suspend fun saveMessage(message: ChatMessage)
- suspend fun updateSessionTitle(sessionId: String, title: String)

ChatRepositoryImpl (data layer)

- Implements ChatRepository
- Delegates local DB operations to ChatSessionDao and ChatMessageDao
- Delegates AI streaming to FirebaseAIDataSource
- Handles the session title auto-generation logic: takes first user message, truncates to 40 chars, calls updateSessionTitle()

---

5. Domain Layer

5.1 Use Cases

Use Cases sit between ViewModels and the Repository. Each use case does exactly one thing. They are plain Kotlin classes with no Android dependencies, making them easy to test.

Use Case Class	Input	Output	Description
GetSessionsUseCase	None	Flow<List<ChatSession>>	Returns all sessions sorted by most recent.
GetMessagesUseCase	sessionId: String	Flow<List<ChatMessage>>	Returns all messages for a session.
CreateSessionUseCase	None	ChatSession	Creates and persists a new empty session.
DeleteSessionUseCase	sessionId: String	Unit	Deletes session and all its messages.
SendMessageUseCase	sessionId, prompt, history, image?	Flow<String>	Saves user message, streams AI response, saves AI message.
UpdateSessionTitleUseCase	sessionId, title	Unit	Updates session title after first message.

ℹ *For a first project, use cases are optional if they feel like overkill. You can call the repository directly from ViewModels and add use cases later. The architecture still holds.*

---

6. Presentation Layer

6.1 UI State Pattern

Every screen has a single sealed class or data class representing its complete UI state. The ViewModel holds one StateFlow of this state. Compose observes it and recomposes only when the state changes.

ChatUiState

- messages: List<ChatMessage> — the full message list for the current session
- isLoading: Boolean — true while the AI response is incoming
- currentStreamingText: String — the partial AI response text being streamed
- error: String? — non-null when an API error occurred
- currentSessionId: String? — the active session's ID

HomeUiState

- suggestions: List<String> — the 4 suggestion chip labels
- recentSessions: List<ChatSession> — for quick-access display

HistoryUiState

- sessions: List<ChatSession> — all sessions
- searchQuery: String — current search filter text
- filteredSessions: List<ChatSession> — derived from sessions filtered by searchQuery

6.2 ViewModel Responsibilities

ChatViewModel

- Loads messages for a given sessionId on init
- On user send: calls SendMessageUseCase, collects the Flow<String>, appends each token to currentStreamingText
- When stream completes: moves currentStreamingText into the messages list as a final ChatMessage
- Handles image attachment: holds a StateFlow<Bitmap?> for the pending image
- Exposes uiState: StateFlow<ChatUiState>

HomeViewModel

- Provides hardcoded or configurable suggestion chip text
- On 'new chat': calls CreateSessionUseCase, emits the new sessionId for navigation

HistoryViewModel

- Collects GetSessionsUseCase flow into uiState
- Handles search: filters sessions list by searchQuery using contains() on title
- Handles delete: calls DeleteSessionUseCase on swipe/long-press

SettingsViewModel

- Reads and writes to SettingsRepository (DataStore backed)
- Exposes selected Gemini model, theme preference

6.3 Composable Screen Structure

Each screen is a stateless Composable that receives state and callbacks. The ViewModel is obtained at the NavGraph level via hiltViewModel() and passed down.

Screen Composable	What It Renders	Key Child Composables
HomeScreen	Greeting, suggestion chips, input bar	SuggestionChip, InputBar
ChatScreen	Message list, streaming bubble, input bar	MessageBubble, StreamingBubble, InputBar, ImagePreview
DrawerContent	Search bar, new chat button, session list	SessionListItem, SearchBar
SettingsScreen	Model picker, API info, about	ModelSelector, SettingsRow

---

7. Complete Data Flow

7.1 User Sends a Message (Happy Path)

This is the most critical flow in the app. Every step must work correctly for AuraChat's core feature to function.

- Step 1 — User types message and taps send in ChatScreen
  - Step 2 — ChatScreen calls onSendMessage(text, bitmap?) callback
  - Step 3 — ChatViewModel receives the call, sets isLoading = true in uiState
  - Step 4 — ViewModel adds a USER ChatMessage to the local list immediately (optimistic UI)
  - Step 5 — ViewModel calls SendMessageUseCase(sessionId, history, prompt, image)
  - Step 6 — SendMessageUseCase calls ChatRepository.streamAIResponse()
  - Step 7 — ChatRepositoryImpl delegates to FirebaseAIDataSource.streamResponse()
  - Step 8 — FirebaseAIDataSource calls Firebase generativeModel.generateContentStream()
  - Step 9 — Firebase SDK emits Flow<GenerateContentResponse> token by token
  - Step 10 — Each token is mapped to a String and emitted up through the Flow chain
  - Step 11 — ChatViewModel collects each String token in a coroutine
  - Step 12 — ViewModel appends token to currentStreamingText in uiState
  - Step 13 — Compose recomposes StreamingBubble with updated text in real time
  - Step 14 — When Flow completes, ViewModel creates a final ChatMessage with full content
  - Step 15 — Final message is saved to Room via ChatMessageDao.insertMessage()

    - Step 16 — isLoading = false, currentStreamingText = cleared

7.2 Loading a Past Session

- User taps a session in DrawerContent
- Drawer calls onSessionSelected(sessionId) callback
- NavController navigates to ChatScreen with sessionId as a route argument
- ChatScreen calls hiltViewModel() which creates a ChatViewModel scoped to this NavBackStackEntry
- ChatViewModel.init: calls GetMessagesUseCase(sessionId), collects Flow into uiState.messages
- ChatScreen renders the full message history

7.3 Image Attachment Flow

- User taps + button in InputBar
- Android photo picker intent is launched
- Selected image URI is decoded to Bitmap in the ViewModel (on IO dispatcher)
- Bitmap is stored in ViewModel state as pendingImage: StateFlow<Bitmap?>
- ImagePreview composable renders the pending image above the input bar
- On send, the Bitmap is passed alongside the text to SendMessageUseCase
- FirebaseAIDataSource includes the Bitmap as an InlineDataPart in the Gemini request
- After send, pendingImage is cleared

---

8. Navigation Architecture

8.1 Route Definitions

AuraChat uses Jetpack Navigation Compose with string-based routes for simplicity. Routes are defined as constants in a Screen sealed class.

Route	Screen	Arguments	Entry Point
home	HomeScreen	None	App launch startDestination
chat/{sessionId}	ChatScreen	sessionId: String	From HomeScreen or Drawer
settings	SettingsScreen	None	From Drawer or TopBar

8.2 Navigation Structure

- Single Activity: MainActivity annotated with @AndroidEntryPoint

- NavHost sits inside a ModalNavigationDrawer so the drawer wraps all screens
- DrawerContent is always available regardless of which screen is active
- ModalNavigationDrawer state is hoisted to MainActivity level
- Each screen gets its own @HiltViewModel scoped to its NavBackStackEntry
- Back stack: ChatScreen → back → HomeScreen (Home is always the root)

8.3 Navigation Events from ViewModels

ViewModels cannot hold NavController references. Navigation events are communicated via a SharedFlow<NavigationEvent> exposed from the ViewModel and collected in the Composable.

- HomeViewModel emits NavigationEvent.ToChatScreen(sessionId) after session creation
- ChatScreen collects this event and calls navController.navigate('chat/$sessionId')

ℹ *Never pass NavController into a ViewModel. This creates a memory leak and breaks the architecture separation.*

---

9. Settings & Persistence

9.1 DataStore for Settings

App preferences (selected Gemini model, theme) are stored using Jetpack DataStore (Preferences). DataStore is the modern replacement for SharedPreferences and is coroutine-native.

- Selected model: String key, default = 'gemini-2.0-flash'
- Available models: ['gemini-2.0-flash', 'gemini-1.5-flash', 'gemini-1.5-pro']
- Theme: currently always dark (v1 only supports dark mode)

9.2 Security Considerations

- No API keys are stored in the app or in code — Firebase AI Logic manages authentication through google-services.json and Firebase project credentials
- google-services.json must NEVER be committed to a public GitHub repository — add it to .gitignore
- Room database is stored in the app's private data directory, not accessible to other apps

---

10. Error Handling Strategy

10.1 Error Categories

Error Type	Where It Occurs	How It's Handled
Network/Firebase error	FirebaseAIDataSource.streamResponse()	Caught, mapped to error string, emitted in Flow

Error Type	Where It Occurs	How It's Handled
Empty AI response	ChatRepositoryImpl	Substituted with 'No response received.' message
Room database error	DAO operations	Caught in Repository, logged, UI shows retry option
Image decode failure	ChatViewModel image picker	pendingImage stays null, user sees toast
Session not found	ChatViewModel init	Navigates back to HomeScreen

10.2 UI Error Display

- Inline error messages appear inside the chat bubble area (not modal dialogs)
- Error state in ChatUiState.error: String? — non-null triggers an error banner below the message list
- Error banner has a 'Retry' button that re-triggers the last SendMessageUseCase call

---

11. Vibe Coding Prompting Guide

When prompting your AI coding agent, always paste the relevant section of this TDD as context. The more context the agent has, the better the output. Below are example prompt templates for each phase.

Phase 1 — Project Setup Prompt Template

ℹ *Paste this into your AI agent when setting up the project for the first time.*

"I am building AuraChat, an Android Jetpack Compose app. Set up the project with the following Gradle configuration: [paste the gradle config from the PRD]. Create the package structure: com.aurachat with sub-packages data, domain, presentation, and ui. Create an Application class called AuraChatApplication annotated with @HiltAndroidApp. Annotate MainActivity with @AndroidEntryPoint. Set up the AuraChat dark theme using Color.kt and Theme.kt with true black (#000000) backgrounds."

Phase 2 — Data Layer Prompt Template

"Using the schema in the AuraChat TDD, create the Room database. Create two entities: ChatSession with fields [id, title, createdAt, updatedAt] and ChatMessage with fields [id, sessionId, role, content, imageUri, timestamp, isStreaming]. Create DAOs for each entity as defined. Create AuraChatDatabase extending RoomDatabase. Create a DatabaseModule Hilt module that provides the database and both DAOs as Singletons."

Phase 3 — Firebase AI Streaming Prompt Template

"Create FirebaseAIDataSource in the data.remote package. It should expose a function streamResponse(history: List<ChatMessage>, prompt: String, imageBitmap: Bitmap?): Flow<String>. Use Firebase AI Logic with GenerativeBackend.googleAI() and model 'gemini-2.0-flash'. Build the chat history as a list of Content objects. Call generateContentStream() and map each GenerateContentResponse to its text. Handle errors by catching exceptions and emitting an error string. Inject this class with Hilt."

Phase 4 — ChatViewModel Streaming Prompt Template

"Create ChatViewModel annotated with @HiltViewModel. It should hold a StateFlow<ChatUiState>. On sendMessage(text, bitmap): set isLoading=true, add a USER message to the list, collect the Flow<String> from ChatRepository.streamAIResponse(), append each token to currentStreamingText in the state, and when the flow completes save the final message to Room and clear isLoading. The ViewModel must never import Firebase, Room, or any data layer class directly."

11.2 Golden Rules for Vibe Coding AuraChat

- Always give the AI agent this TDD + the PRD as context before asking it to write code
- Build and test each Phase before starting the next one
- If the AI puts network or database code in a Composable, reject it and re-prompt
- If Hilt fails to compile, check: @HiltAndroidApp on Application, @AndroidEntryPoint on Activity, @HiltViewModel on ViewModels
- Never accept the first pass — always ask the agent to review its own code for architecture violations
- If streaming doesn't work, isolate FirebaseAIDataSource first with a simple test before wiring it to the ViewModel

---

AuraChat TDD — End of Document